Deliverable 2 — Design Document

Healthcare Web Application

Course Code — CSE 616
Course Name — Web Engineering Lab

Group Number: 14

Group Members:
Israt Zannat — 22701027
Sajid Shahriar Sanam — 22701031
Shajeda Isnat — 22701060

Course Instructor:
Professor Rokan Uddin Faruqui

Department of Computer Science and Engineering,
University of Chittagong

Date Prepared: October 19, 2025

Contents

1 System Architecture **3**
1.1 Overview . . . 3
1.2 Supporting Services . . . 3
1.3 Communication Flow . . . 3
1.4 Key Design Benefits . . . 4

2 Database Design and ER Diagram **4**
2.1 Overview . . . 5
2.2 Entities and Attributes . . . 6
2.3 Relationships Overview . . . 9
2.4 Design Considerations . . . 9

3 API Specification and Endpoints **10**
3.1 Introduction . . . 10
3.2 API Endpoint Overview . . . 10
3.3 Example Request and Response . . . 11
3.4 API Security Overview . . . 12
3.5 API Communication Flow . . . 12
3.6 Summary . . . 12

4 UI/UX Mockups **13**
4.1 Landing Page . . . 13
4.2 Login and Sign Up Pages . . . 14
4.3 Dashboard Interfaces . . . 15
4.4 Design Highlights . . . 16

5 Security Architecture **17**
5.1 Overview . . . 17
5.2 Core Security Objectives . . . 17
5.3 Authentication and Authorization . . . 17
5.4 Data Protection and Privacy . . . 18
5.5 Application and Network Security . . . 18
5.6 Logging and Monitoring . . . 18
5.7 Backup and Recovery . . . 18
5.8 Security Architecture Summary . . . 19
5.9 Future Security Enhancements . . . 19

1 System Architecture

The **Healthcare Web Application** follows a modular three-tier architecture that ensures scalability, data security, and efficient role-based functionality. The system integrates a React-based frontend, Node.js/Express backend, and a MongoDB database, connected through RESTful APIs over HTTPS. This structure provides a robust foundation for handling healthcare operations such as patient management, appointment scheduling, and prescription generation.

1.1 Overview

The architecture is designed to separate responsibilities into distinct layers:

- **Frontend (Client Layer):** Developed using *React.js*, the Single Page Application (SPA) handles user interactions for patients, doctors, and admins. It communicates with the backend via RESTful API calls and provides a responsive, accessible interface.
- **Backend (Application Layer):** Implemented with *Node.js* and *Express*, the backend manages authentication, authorization, and application logic. It also handles all communication with the database and external services.
- **Database (Data Layer):** Uses *MongoDB* for storing structured and semi-structured healthcare data, including users, appointments, and prescriptions. Mongoose ORM ensures consistency and schema validation.

1.2 Supporting Services

The system architecture also integrates several key support modules to ensure smooth operation:

- **Authentication Service:** Utilizes JWT (JSON Web Token) for secure access and refresh token management.
- **File Storage:** Manages uploaded medical documents and images, stored either locally or in the cloud.
- **Email / Notification Service:** Implements Nodemailer or SendGrid for sending notifications such as appointment confirmations and password resets.
- **Admin Dashboard:** Provides administrators with tools for user management, analytics, and system reporting.

1.3 Communication Flow

- Users (patients, doctors, admins) interact with the system via HTTPS requests.
- The frontend sends JSON-based REST API calls to the backend.
- The backend verifies JWT tokens, processes logic, and queries MongoDB.
- File uploads and email notifications are processed through external modules.

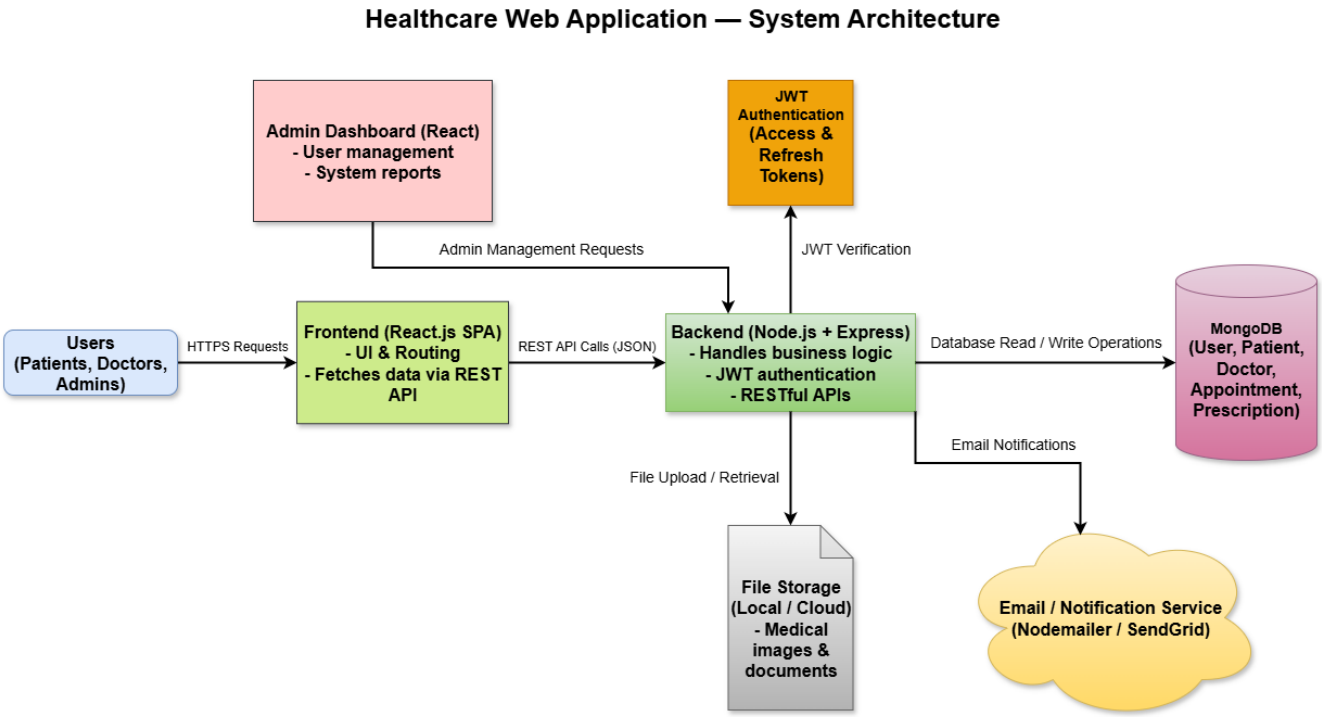

Figure 1: System Architecture of the Healthcare Web Application. The diagram illustrates secure communication between the React frontend and the Node.js/Express backend using RESTful APIs over HTTPS. The backend handles authentication through JWT, manages data stored in MongoDB, supports file storage for medical documents, and integrates an email service for notifications. An admin dashboard provides management and reporting capabilities.

1.4 Key Design Benefits

- **Scalability:** Each layer can be independently scaled or containerized using Docker.
- **Security:** All API endpoints are protected with JWT authentication and HTTPS encryption.
- **Maintainability:** Modular architecture simplifies debugging, updates, and team collaboration.
- **Reusability:** Common services like authentication and file storage are reusable across different modules.

2 Database Design and ER Diagram

The database design of the **Healthcare Web Application** ensures reliable data storage, integrity, and scalability to support multiple user roles (patients, doctors, and administrators). The system uses a **MongoDB** NoSQL database due to its flexible document-oriented structure, which is ideal for storing healthcare records with variable schemas such as medical history, prescriptions, and documents.

2.1 Overview

The database is designed around the central *Users* collection, from which patient, doctor, and admin profiles are extended. Supporting collections such as *Appointments*, *Prescriptions*, *Documents*, and *Notifications* capture the core healthcare workflows, while *SystemLogs* and *Feedback* provide operational transparency and performance evaluation.

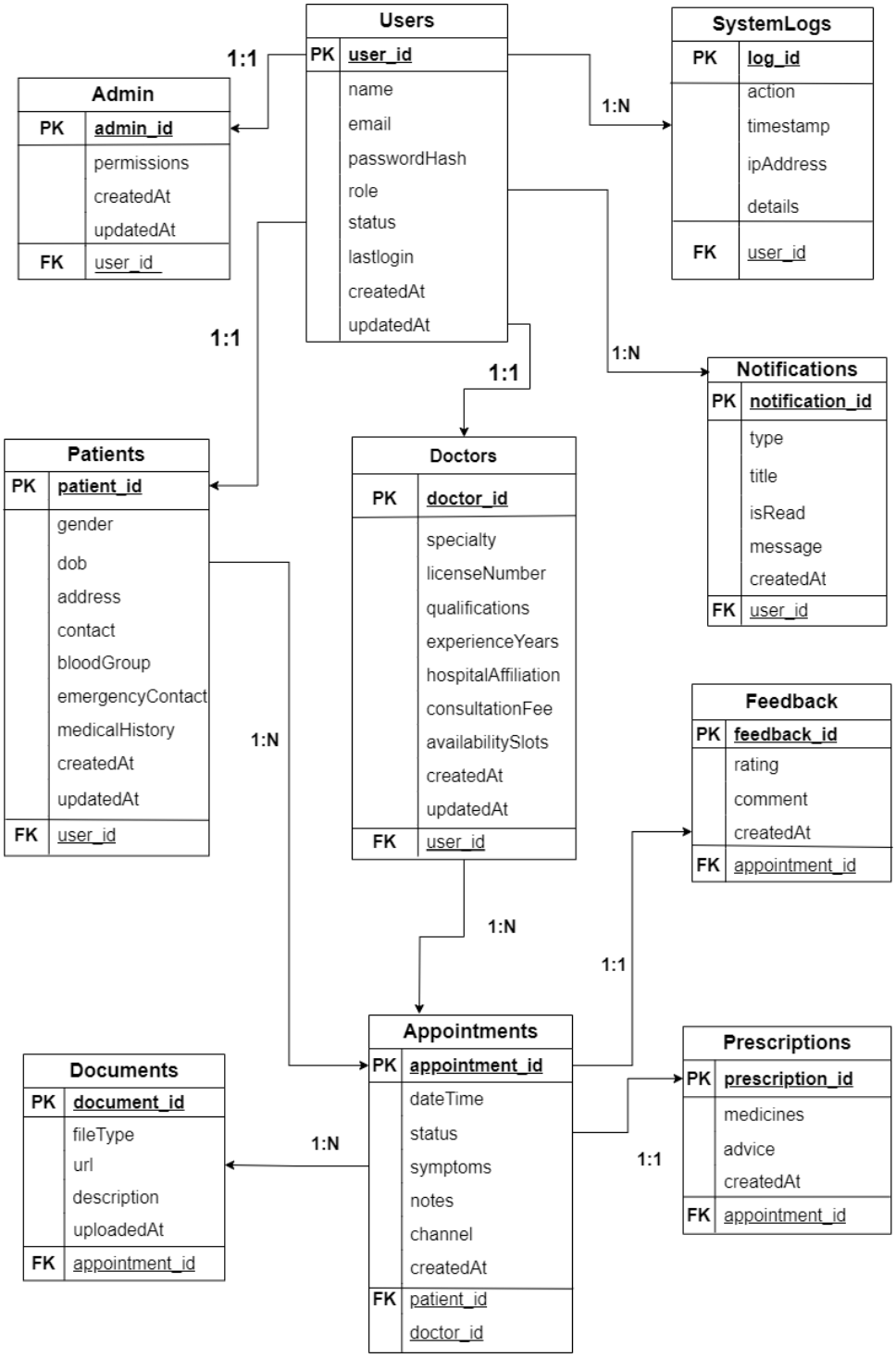

Figure 2: Entity–Relationship Diagram (ERD) of the Healthcare Web Application showing key entities, attributes, and relationships among various entities.

2.2 Entities and Attributes

The following tables describe the entities, their attributes, and their relationships in the database schema.

1. Users

Purpose: Central entity representing all system users.

Field	Type	Description
user_id (PK)	UUID/ObjectId	Unique identifier for user
name	String	Full name of the user
email	String (unique)	Used for authentication
passwordHash	String	Hashed password for security
role	Enum(admin, doctor, patient)	Defines the user's system role
status	Enum(active, inactive)	Indicates account status
lastLogin	DateTime	Timestamp of last login
createdAt, updatedAt	DateTime	Record timestamps

4. Admins

Purpose: Stores administrative control information.

Field	Type	Description
admin_id (PK)	UUID/ObjectId	Unique admin identifier
user_id (FK)	UUID → Users.user_id	Links to user table (1:1)
permissions	JSON/Text	Defines system privileges
createdAt, updatedAt	DateTime	Record timestamps

2. Patients

Purpose: Stores patient-specific information.

Field	Type	Description
patient_id (PK)	UUID/ObjectId	Unique patient identifier
user_id (FK)	UUID → Users.user_id	Links to user table (1:1)
dob	Date	Date of birth
gender	String	Gender
bloodGroup	String	Blood type
contact	String	Contact number
address	Text	Address
emergencyContact	String	Emergency contact info
medicalHistory	JSON/Text	Previous medical conditions
createdAt, updatedAt	DateTime	Record timestamps

3. Doctors

Purpose: Contains doctor professional details.

Field	Type	Description
doctor_id (PK)	UUID/ObjectId	Unique doctor identifier
user_id (FK)	UUID → Users.user_id	Links to user table (1:1)
specialty	String	Area of specialization
licenseNumber	String	Medical license number
qualifications	Text	Professional degrees/certificates
experienceYears	Integer	Years of experience
hospitalAffiliation	String	Associated hospital or clinic
consultationFee	Decimal	Fee charged per appointment
availabilitySlots	JSON/Text	Available times for consultation
createdAt, updatedAt	DateTime	Record timestamps

5. Appointments

Purpose: Core entity connecting patients and doctors.

Field	Type	Description
appointment_id (PK)	UUID/ObjectId	Unique appointment record
patient_id (FK)	UUID → Patients.patient_id	Linked patient
doctor_id (FK)	UUID → Doctors.doctor_id	Linked doctor
dateTime	DateTime	Scheduled time
status	Enum(pending, confirmed, cancelled, completed)	Appointment status
symptoms	Text	Patient-described symptoms
notes	Text	Doctor notes
channel	Enum(online, in-person)	Appointment mode
createdAt	DateTime	Creation timestamp

6. Prescriptions

Purpose: Stores doctor prescriptions.

Field	Type	Description
prescription_id (PK)	UUID/ObjectId	Unique prescription record
appointment_id (FK)	UUID → Appointments.appointment_id	Related appointment
medicines	JSON/Text	List of medicines (name, dosage, duration)
advice	Text	Additional medical guidance
createdAt	DateTime	Record timestamp

7. Documents

Purpose: Stores uploaded medical reports.

Field	Type	Description
document_id (PK)	UUID/ObjectId	Unique document record
appointment_id (FK)	UUID → Appointments.appointment_id	Linked appointment
fileType	String	Type of file (PDF, JPG, etc.)
url	String	File storage path or URL
description	Text	Document description
uploadedAt	DateTime	Upload timestamp

8. Notifications

Purpose: Manages system messages and alerts.

Field	Type	Description
notification_id (PK)	UUID/ObjectId	Unique notification identifier
user_id (FK)	UUID → Users.user_id	Recipient of notification
type	Enum(appointment, reminder, system)	Notification category
title	String	Notification title
message	Text	Content of notification
isRead	Boolean	Read/unread flag
createdAt	DateTime	Created timestamp

9. Feedback

Purpose: Stores user feedback post-appointment.

Field	Type	Description
feedback_id (PK)	UUID/ObjectId	Unique feedback record
appointment_id (FK)	UUID → Appointments.appointment_id	Related appointment
rating	Integer (1–5)	Rating given by patient
comment	Text	Feedback text
createdAt	DateTime	Submission timestamp

10. SystemLogs

Purpose: Tracks system events for auditing.

Field	Type	Description
log_id (PK)	UUID/ObjectId	Unique log entry
user_id (FK)	UUID → Users.user_id	User associated with the event
action	String	Type of system action performed
timestamp	DateTime	Time of action
ipAddress	String	Source IP of request
details	Text	Additional event data

2.3 Relationships Overview

- User ↔ Patient — 1:1
- User ↔ Doctor — 1:1
- User ↔ Admin — 1:1
- Doctor ↔ Appointments — 1:N
- Patient ↔ Appointments — 1:N
- Appointment ↔ Prescription — 1:1
- Appointment ↔ Feedback — 1:1
- Appointment → Documents — 1:N
- User ↔ Notifications — 1:N
- User ↔ SystemLogs — 1:N

2.4 Design Considerations

- Each user is stored once in the *Users* collection, maintaining data normalization.
- Subcollections (*Patients*, *Doctors*, *Admins*) store role-specific attributes.
- Cascading relationships ensure referential integrity between appointments, prescriptions, and feedback.
- Timestamps on every record support auditing and analytics.

3 API Specification and Endpoints

3.1 Introduction

The API layer of the **Healthcare Web Application** acts as a bridge between the *React.js frontend* and the *Node.js + Express backend*, with *MongoDB* as the central database. All endpoints follow RESTful principles, exchange data in JSON format, and are secured using **JWT (JSON Web Tokens)** with role-based access for Patients, Doctors, and Admins.

The API is designed to ensure scalability, maintainability, and data security through HTTPS, validation, and modular route organization.

3.2 API Endpoint Overview

The following tables summarize the key API modules and their main operations.

Authentication & Authorization

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a new user (Patient/Doctor/Admin)	Public
POST	/api/auth/login	Authenticate user and return JWT token	Public
POST	/api/auth/reset-password	Reset password via token	Public

Patient Management

Method	Endpoint	Description	Access
POST	/api/patients	Create or update patient profile	Auth: Patient
GET	/api/patients/:id	Fetch patient profile by ID	Auth: Doctor/Admin
PUT	/api/patients/:id	Edit patient information	Auth: Patient

Doctor Management

Method	Endpoint	Description	Access
POST	/api/doctors	Register doctor profile	Auth: Admin
GET	/api/doctors	List available doctors	Public
PUT	/api/doctors/:id/availability	Update available slots	Auth: Doctor

Appointments

Method	Endpoint	Description	Access
POST	/api/appointments	Book appointment	Auth: Patient
GET	/api/appointments?userId	View appointments by user	Auth: Patient/Doctor/Admin
PUT	/api/appointments/:id/status	Update appointment status	Auth: Doctor/Admin

Prescriptions

Method	Endpoint	Description	Access
POST	/api/prescriptions	Create prescription after consultation	Auth: Doctor
GET	/api/prescriptions/:patientId	View prescriptions by patient	Auth: Patient/-Doctor

Documents

Method	Endpoint	Description	Access
POST	/api/documents/upload	Upload medical reports or images	Auth: Patient/Doctor
GET	/api/documents/:ownerId	Retrieve uploaded documents	Auth: Doctor/Patient

Notifications

Method	Endpoint	Description	Access
GET	/api/notifications/:userId	Fetch notifications for user	Auth: All
PUT	/api/notifications/:id/read	Mark notification as read	Auth: All

Admin Management

Method	Endpoint	Description	Access
GET	/api/admin/users	List all registered users	Auth: Admin
DELETE	/api/admin/users/:id	Remove user account	Auth: Admin

3.3 Example Request and Response

Example 1 — User Login

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "nazimahmed@gmail.com",
  "password": "123456"
}

Response:
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "U12345",
    "name": "Shahriar Nazim",
    "role": "patient"
  }
}
```

Example 2 — Book Appointment

```
POST /api/appointments
Authorization: Bearer <JWT_TOKEN>

{
  "doctorId": "D56789",
  "datetime": "2025-10-12T10:00:00Z",
  "symptoms": "High fever, fatigue",
  "channel": "in-person"
}

Response:
{
  "message": "Appointment booked successfully",
  "appointmentId": "A101"
}
```

3.4 API Security Overview

- **JWT Authentication:** Valid token required for protected routes.
- **Role-Based Access:** Patient, Doctor, Admin permissions separated.
- **Validation & HTTPS:** Ensures safe, consistent communication.
- **CORS Policy:** Only frontend-origin requests are allowed.

3.5 API Communication Flow

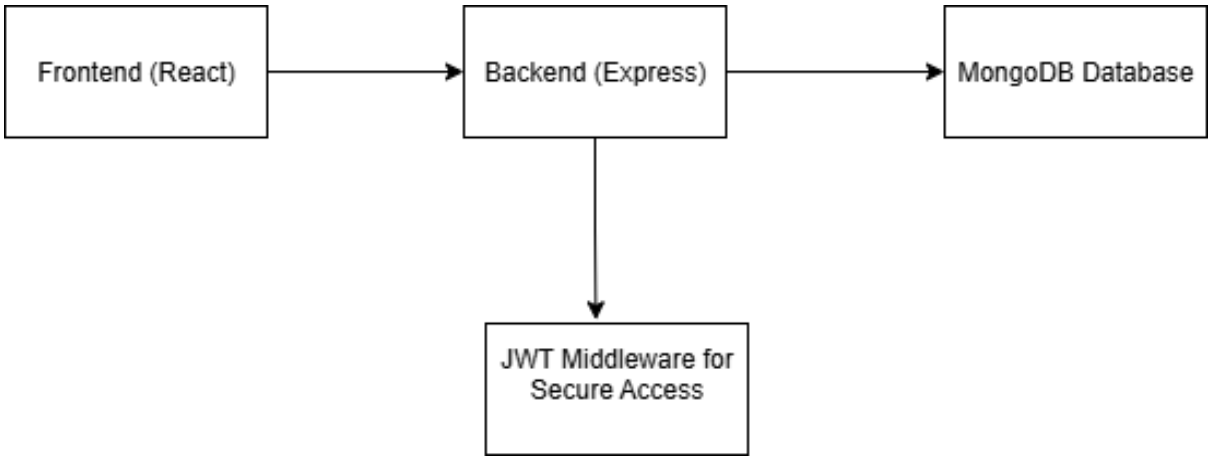

Figure 3: Simplified API communication flow between frontend, backend, and database.

3.6 Summary

This RESTful API architecture enables seamless communication between frontend and backend. Each endpoint is modular, secure, and designed for future scalability — supporting features like telemedicine, analytics, and real-time notifications.

4 UI/UX Mockups

This section presents the user interface and user experience design for the **Healthcare Web Application**. The mockups were designed using *Figma*, focusing on a modern, clean, and responsive layout suitable for Patients, Doctors, and Admins. The goal was to ensure intuitive navigation, visual clarity, and role-specific functionality while maintaining accessibility and usability standards.

The system's key UI modules include:

- **Landing Page:** Introduces the platform, its purpose, and allows quick navigation to login or signup for different user roles.
- **Authentication Pages:** Secure and simple forms for user login and registration, supporting role-based access (Patient, Doctor, Admin).
- **Dashboards:** Dedicated dashboards for each user type — providing a personalized overview of appointments, documents, and reports.

4.1 Landing Page

Purpose: The landing page welcomes users with a modern healthcare theme, describing platform benefits and providing quick access to login or signup.

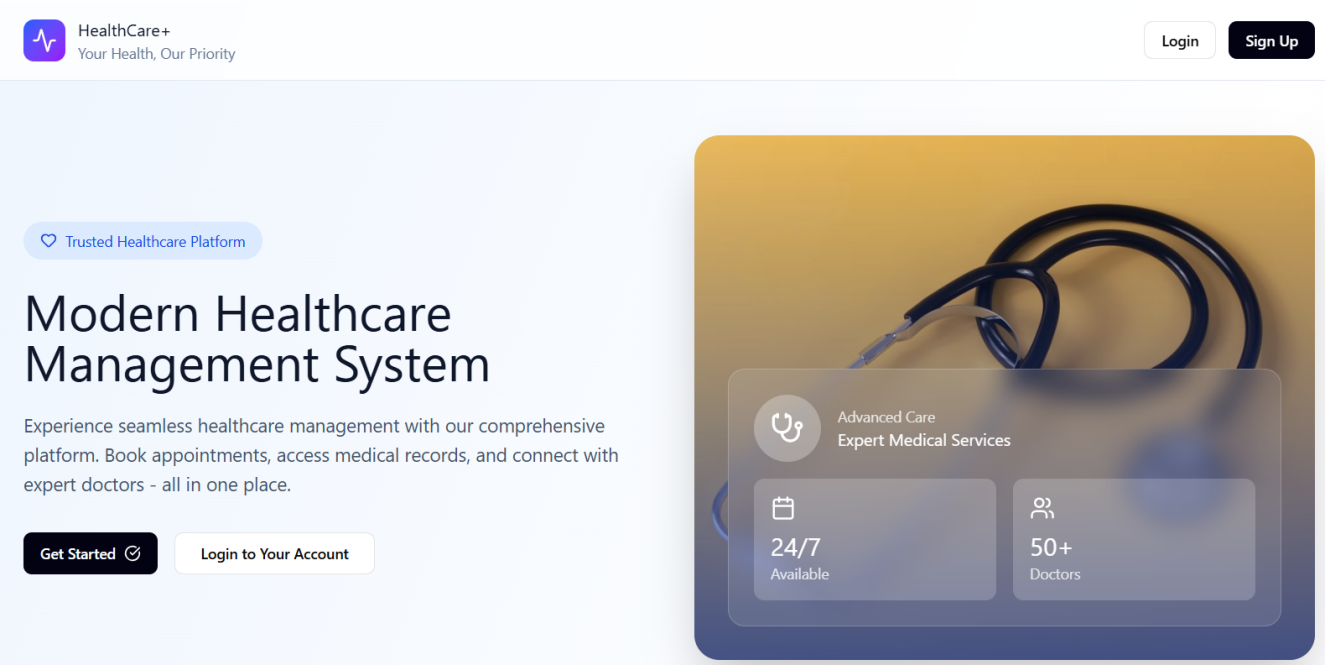

Figure 4: Landing Page — Healthcare+ Platform introducing key features and navigation.

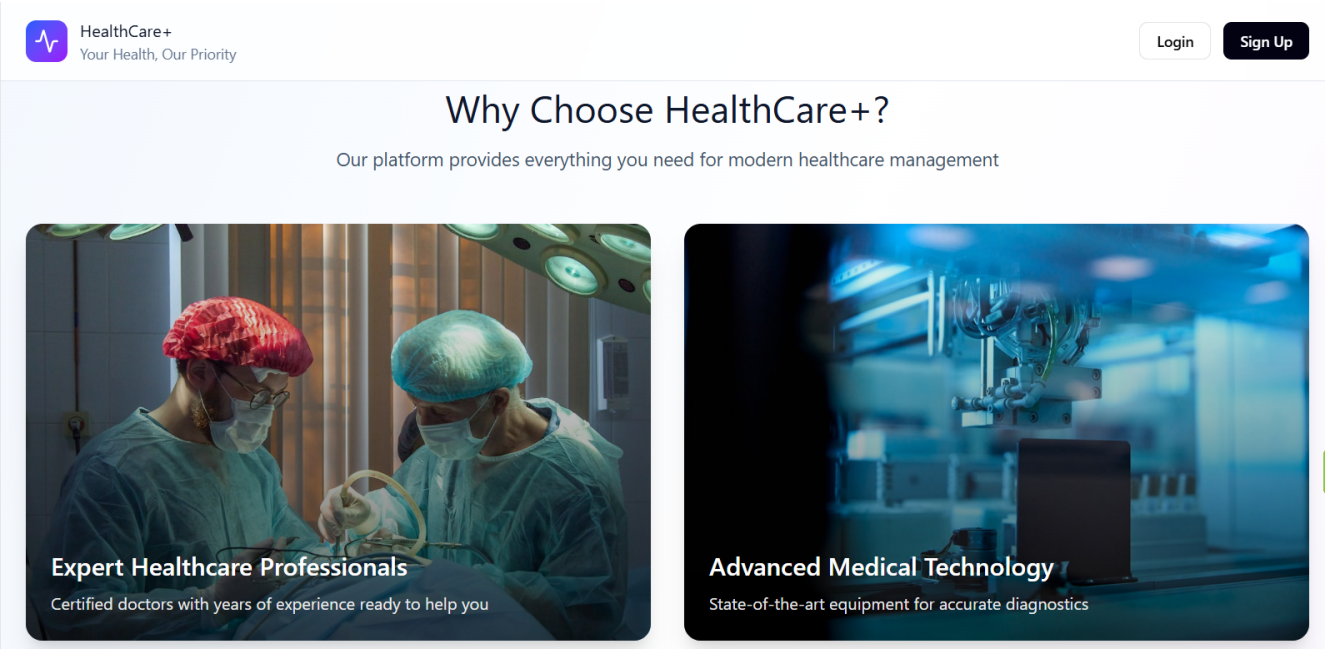

Figure 5: Landing Page — Healthcare+ Platform introducing key features and navigation.

4.2 Login and Sign Up Pages

Purpose: Provides authentication interfaces for all users with input validation, password security, and responsive design.

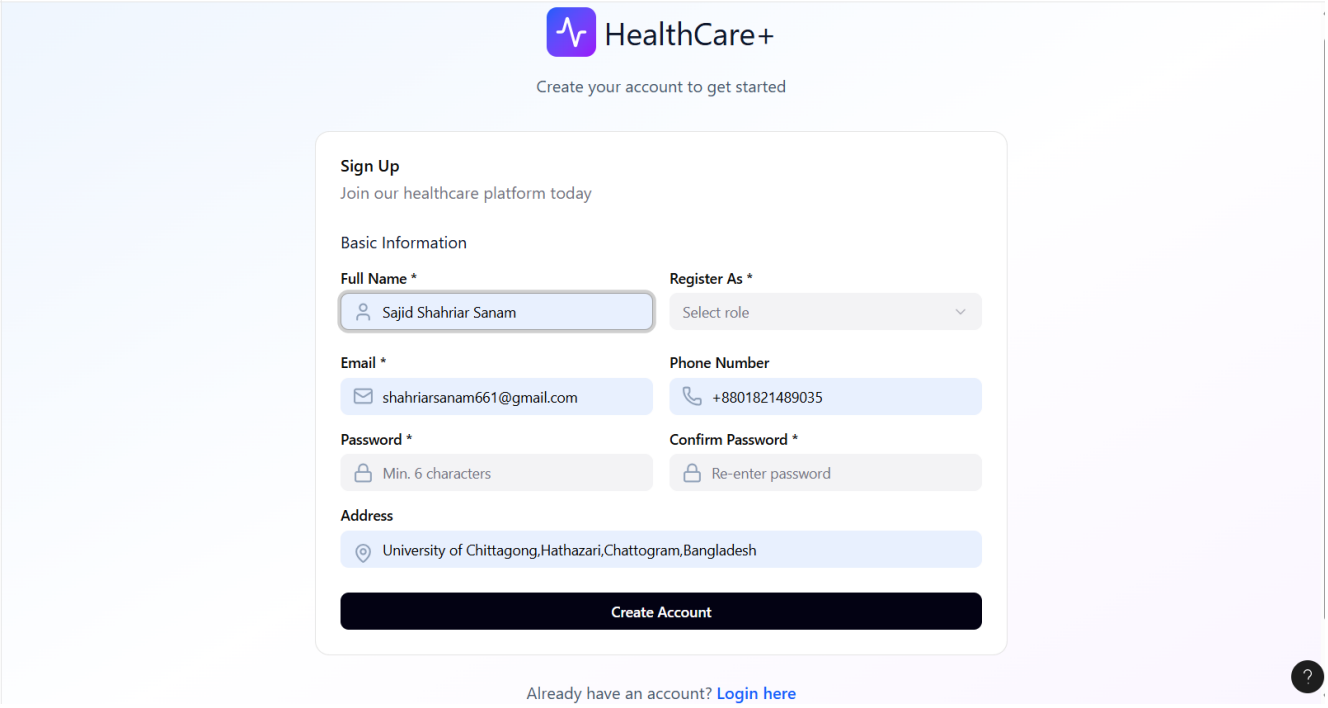

Figure 6: Sign Up Page

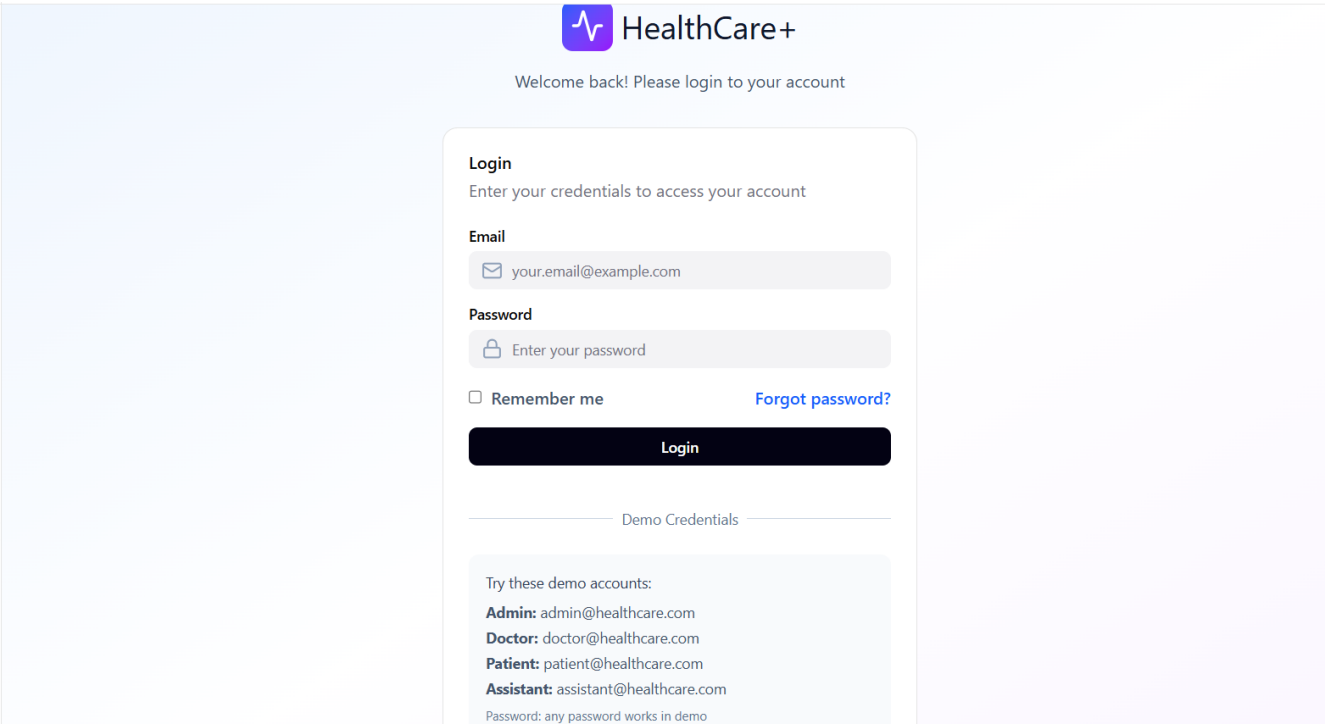

Figure 7: Login Page

4.3 Dashboard Interfaces

Purpose: Dashboards serve as role-specific control panels — Patients can view appointments, prescriptions, and medical reports; Doctors can manage schedules and consultations; Admins can oversee users and system metrics.

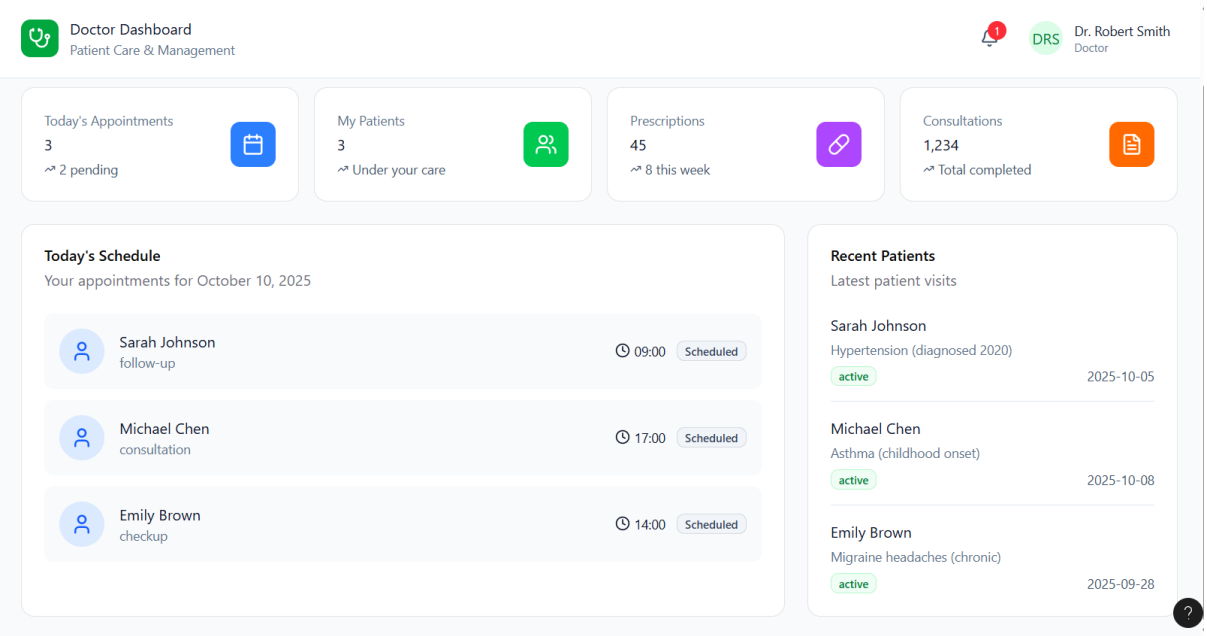

Figure 8: Dashboard — Interfaces for Doctor with essential management features.

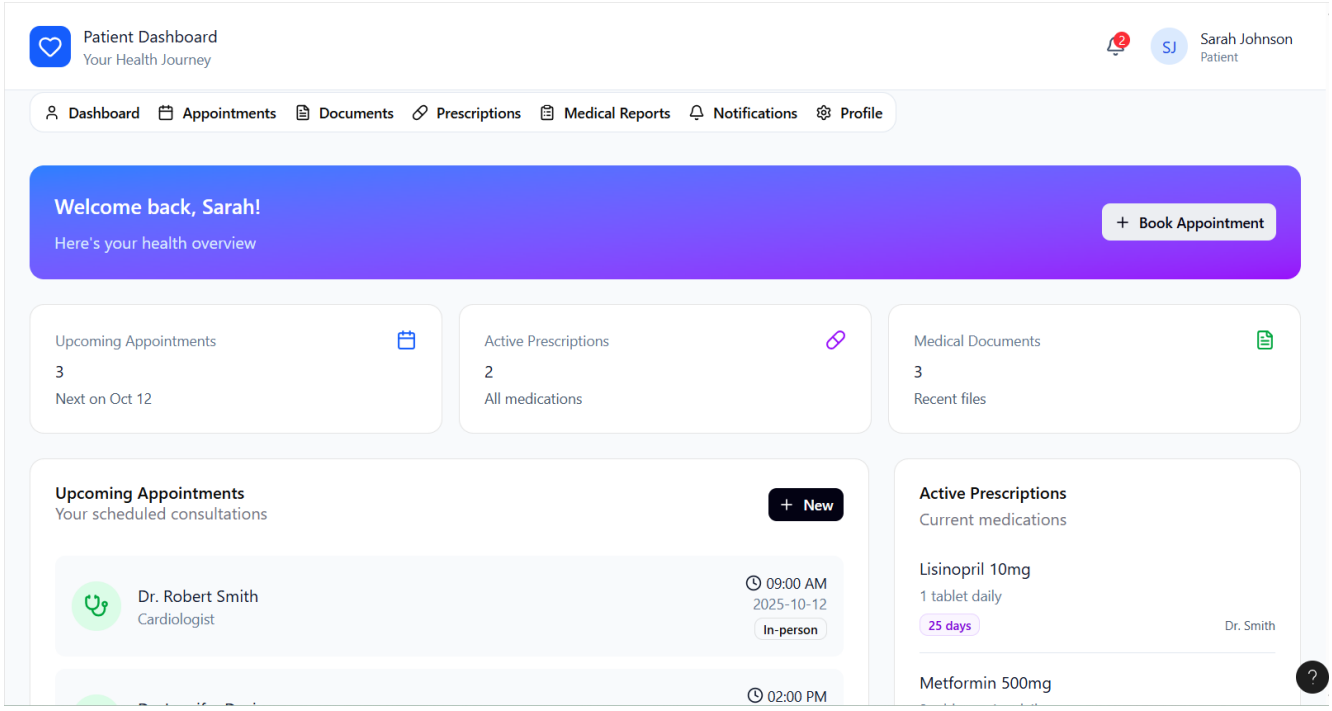

Figure 9: Dashboard — Interface for Patient with essential management features.

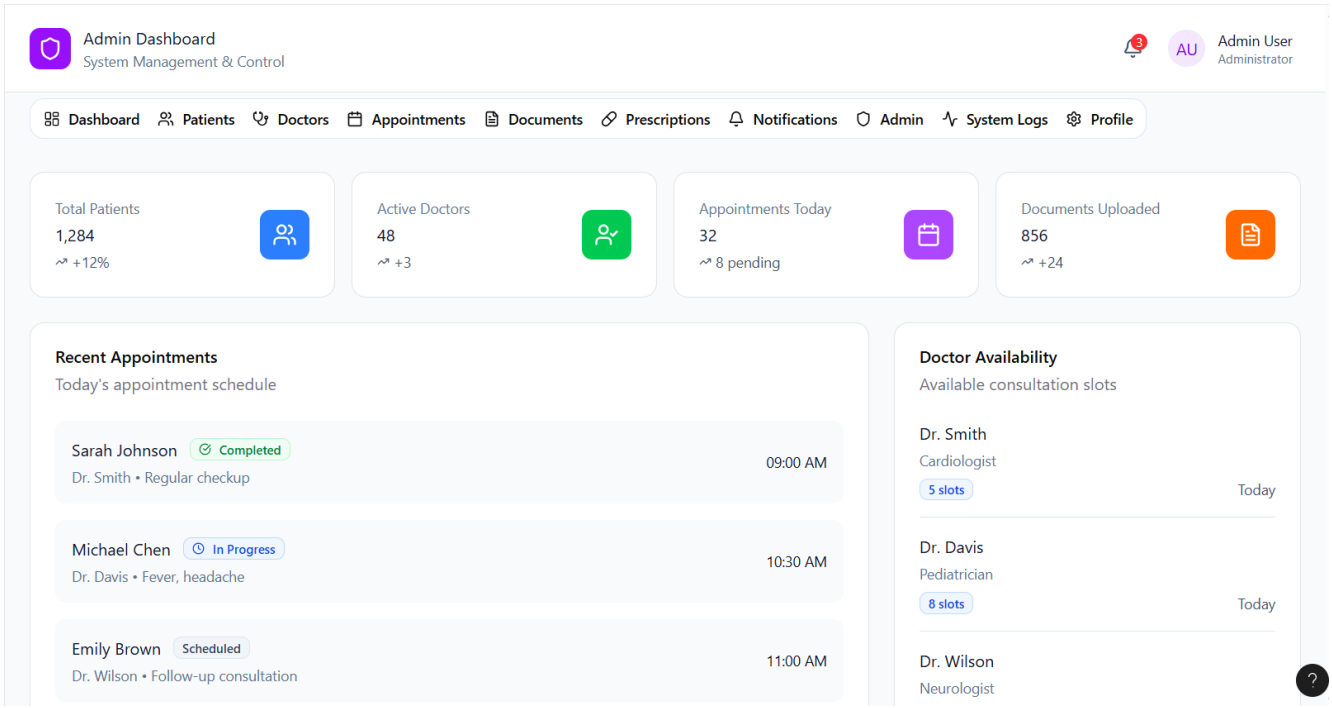

Figure 10: Dashboard — Interface for Admin with essential management features.

4.4 Design Highlights

- **Color Palette:** Professional blue and white tones representing healthcare trust and simplicity.

- **Typography:** Clean, modern fonts ensuring readability and accessibility.
- **Consistency:** Reusable UI components (buttons, forms, cards) for maintainable design.
- **Responsiveness:** All pages are designed to be responsive and adaptive to different devices.
- **Accessibility:** Proper contrast ratios and form feedback elements to support inclusive use.

Summary: The UI/UX mockups effectively visualize the final user experience of the Healthcare Web Application, providing a cohesive, user-friendly, and professional interface aligned with modern web application standards.

5 Security Architecture

5.1 Overview

Security is a critical component of the Healthcare Web Application due to the sensitivity of patient and medical data. The system applies a multi-layered security model to protect the confidentiality, integrity, and availability (CIA) of all information. The key security layers include authentication and authorization, data protection, network security, and monitoring.

5.2 Core Security Objectives

- **Confidentiality:** Only authorized users (Patients, Doctors, and Admins) can access protected data.
- **Integrity:** Medical records, prescriptions, and appointment data cannot be modified without permission.
- **Availability:** The system remains operational and accessible under normal conditions.
- **Accountability:** All significant activities are logged and traceable through audit logs.

5.3 Authentication and Authorization

User identity and access are enforced through token-based authentication and role-based authorization:

- **JWT (JSON Web Token):** After login, the backend issues a signed JWT to the client. Each protected API request includes this token for verification.
- **Role-Based Access Control (RBAC):** Permissions differ by user type:
  - **Patient:** Can manage own profile, view prescriptions, and schedule appointments.

  - **Doctor:** Can view assigned patients, update medical notes, and issue prescriptions.
  - **Admin:** Can manage all users, monitor activity, and view analytics.

- **Password Protection:** All passwords are hashed using a one-way cryptographic algorithm (bcrypt) before storage.

5.4 Data Protection and Privacy

- **Data in Transit:** All communication between frontend and backend occurs over HTTPS to prevent data interception.
- **Data at Rest:** Sensitive data in the database and uploaded medical documents are stored in encrypted form.
- **File Security:** Uploaded files (lab reports, prescriptions) are validated for type and size before storage. Only authorized users can access files via secure URLs.
- **Minimal Data Principle:** Only essential user and medical data are collected and retained.

5.5 Application and Network Security

- **CORS Policy:** The backend accepts requests only from trusted frontend origins.
- **Rate Limiting:** Login and sensitive API endpoints are rate-limited to prevent brute-force attacks.
- **Secure Headers:** Implemented through HTTP security headers (using tools such as Helmet in Node.js).
- **Input Validation:** Every API endpoint validates incoming data to prevent injection attacks (XSS, NoSQL injection).

5.6 Logging and Monitoring

- Important activities such as user login, file uploads, and admin operations are recorded in the *SystemLogs* collection.
- Logs include timestamp, user ID, and action details to support auditing and debugging.
- Regular monitoring helps detect unusual activities, such as failed login attempts or unauthorized access.

5.7 Backup and Recovery

- The MongoDB database is backed up periodically, and backups are stored securely in a separate environment.
- Disaster recovery procedures ensure that critical patient and appointment data can be restored quickly in case of system failure.

5.8 Security Architecture Summary

Figure 11 illustrates the overall security workflow of the system.

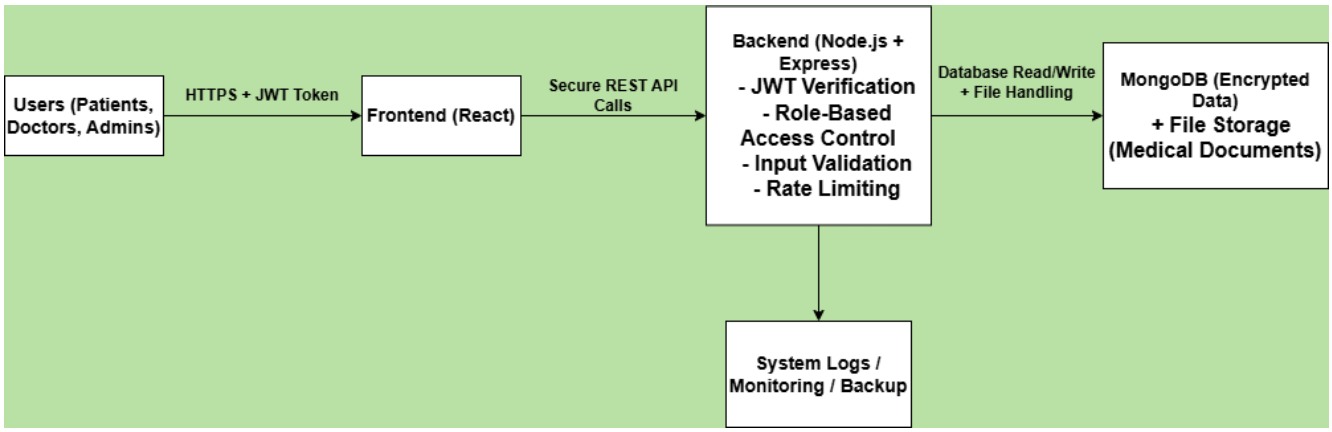

Figure 11: Security Architecture Overview — illustrating authentication flow, HTTPS protection, and secure data handling across layers.

The architecture ensures that every interaction between the React frontend, Node.js backend, and MongoDB database passes through secure, validated, and authorized channels. These practices align with modern web-application security standards and provide a safe environment for managing sensitive healthcare information.

5.9 Future Security Enhancements

To further strengthen the system in upcoming versions, the following improvements are planned:

- **Two-Factor Authentication (2FA):** Integrate an additional authentication step through email or SMS to enhance login security.
- **End-to-End Encrypted Telemedicine Sessions:** Ensure that doctor-patient virtual consultations are encrypted using WebRTC-based secure channels.
- **Regular Security Audits:** Conduct periodic vulnerability assessments and penetration testing to identify and fix security loopholes.
- **Automated Threat Detection:** Implement AI-based anomaly detection to monitor unauthorized access or suspicious user activity.
- **Data Retention Policy:** Introduce automatic data lifecycle management to safely archive or delete outdated medical records.

These enhancements will ensure that the Healthcare Web Application remains compliant with modern security standards and continuously improves its defense mechanisms.